

Exercises week 1

prof. Richard Torkar

July 18, 2025

Es ist nicht genug, zu wissen, man muß auch anwenden; es ist nicht genug, zu wollen, man muß auch tun.

Johann Wolfgang von Goethe

YOU WANT TO DO THIS STEP-BY-STEP to continuously learn things. There are simply too many concepts that we will study in this course for you to learn everything the week before the exam.

Every week has exercises; you do not need to submit them, but doing them will allow you to get a better understanding of what we teach in this course. Many exercises are based on McElreath's exercises, which he has online.¹ I will pick the ones I believe are the most important, with solutions! Each document will introduce, first, the exercises and then the solutions for all exercises.

¹ https://github.com/rmcelreath/stat_rethinking_2020/tree/main/homework

EXERCISE 1. If you haven't done it, please install all software you will use in this course! You have two options: either you setup the required environment locally, or containerized. For a local setup, conduct the following steps. For a containerized setup, see below.

1. Install R v4.* or later for your platform.²
2. Install an IDE for your platform, e.g., RStudio.³ However, I personally use Visual Code most of time since RStudio is a bit bloated in my humble opinion.⁴
3. Follow the instructions and configure your tool-chain (i.e., compiler etc.) for your operating system (i.e., Windows^{5,6}, Mac⁷, or Linux⁸). Make sure to read the instructions **before** you start doing anything. **Important:** It's essential that you follow the correct instructions below, e.g., for Windows if you use R v4.4 you should use RTools44.
4. Restart RStudio and then follow the instructions starting with **installation of RStan**.⁹
5. We will use the latest version of the Stan compiler and sampler. After having restarted RStudio please execute the following commands in the RStudio's console:

```
install.packages(c("coda", "mvtnorm", "devtools", "loo", "dagitty", "shape"))
devtools::install_github("stan-dev/cmdstanr")
cmdstanr::install_cmdstan()
```

² <https://ftp.acc.umu.se/mirror/CRAN/>

³ <https://www.rstudio.com/products/rstudio/download/#download>

⁴ <https://code.visualstudio.com/docs/languages/r>

⁵ <https://github.com/stan-dev/rstan/wiki/Configuring-C---Toolchain-for-Windows>

⁶ For R version 4.4+, see <https://cran.rstudio.com/bin/windows/Rtools/>

⁷ <https://github.com/stan-dev/rstan/wiki/Configuring-C---Toolchain-for-Mac>

⁸ <https://github.com/stan-dev/rstan/wiki/Configuring-C-Toolchain-for-Linux>

⁹ <https://github.com/stan-dev/rstan/wiki/RStan-Getting-Started>

6. Finally, we install the `rethinking` package which is the package we'll use in this course to declare our statistical models,

```
devtools::install_github("rmcelreath/rethinking")
```

Alternatively, you can use a containerized setup. There are three ways of approaching this:

1. **Local devcontainer** (i.e., working within a locally built devcontainer):
 - Install Docker¹⁰ for your system
 - Copy the specification of a containerized environment¹¹ as a Devcontainer
 - Start the environment as a devcontainer¹²
2. **Hosted devcontainer**: host the abovementioned devcontainer with GitHub codespaces¹³

¹⁰ <https://www.docker.com/>

¹¹ For example: <https://github.com/torkar/devcontainer-Stan>

¹² <https://code.visualstudio.com/docs/devcontainers/containers>

¹³ <https://docs.github.com/en/codespaces>

EXERCISE 2. Recall the globe tossing model from the chapter. Compute and plot the grid approximate posterior distribution for each of the following sets of observations. In each case, assume a uniform prior for p .

1. W,W,W
2. W,W,W,L
3. L,W,W,L,W,W,W

EXERCISE 3. Now assume a prior for p that is equal to zero when $p < 0.5$ and is a positive constant when $p \geq 0.5$. Again compute and plot the grid approximate posterior distribution for each of the sets of observations in the problem just above.

SOLUTION 1. I do hope everything is set up correctly now! If not, talk to me or the teaching assistant!

SOLUTION 2. Since the prior is uniform, it can be omitted from the calculations. But I'll show it here, for conceptual completeness. To compute the grid approximate posterior distribution for (1):

```
p_grid <- seq( from=0 , to=1 , length.out=100 )
# likelihood of 3 water in 3 tosses
likelihood <- dbinom( 3 , size=3 , prob=p_grid )
prior <- rep(1,100) # uniform prior
posterior <- likelihood * prior
posterior <- posterior / sum(posterior) # standardize
```

And `plot(posterior)` will produce a simple and ugly plot. This will produce something with nicer labels and a line instead of individual points:

```
plot( posterior ~ p_grid , type="l" )
```

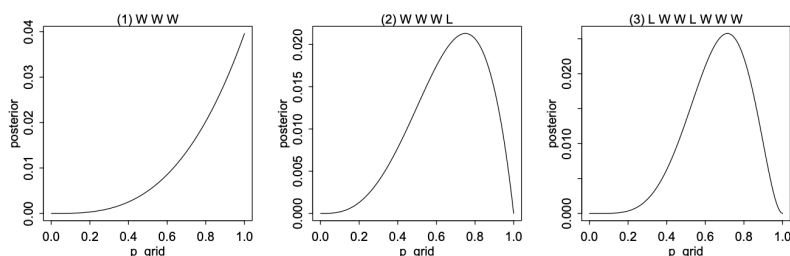
The other two data vectors are completed the same way, but with different likelihood calculations. For (2):

```
# likelihood of 3 water in 4 tosses
likelihood <- dbinom( 3 , size=4 , prob=p_grid )
```

And for (3):

```
# likelihood of 5 water in 7 tosses
likelihood <- dbinom( 5 , size=7 , prob=p_grid )
```

And this is what each plot should look like:



SOLUTION 3. Only the prior has to be changed. For the first set of observations, W W W, this will complete the calculation and plot the result:

```
p_grid <- seq(from = 0 , to = 1 , length.out = 100)
likelihood <- dbinom(3, size = 3, prob = p_grid)
prior <- ifelse(p_grid < 0.5, 0, 1) # new prior
posterior <- likelihood * prior
posterior <- posterior / sum(posterior) # standardize
plot(posterior ~ p_grid , type="l")
```

The other two plots can be completed by changing the likelihood, just as in the previous problem. Here are the new plots, demonstrating that the prior merely truncates the posterior distribution below 0.5:

